

CSCI 4170 - Project in AI and ML

Homework 5 - Summer 2025

Haruto Suzuki

Due date: 11:59 pm, July. 31st

Homework 5 (100 points) CNNs, AEs, GANs, Attention Mechanism

In your project, you will pick an image dataset to solve a classification task. Provide a link to your dataset. You may pick any dataset except MNIST, CIFAR or ImageNet.

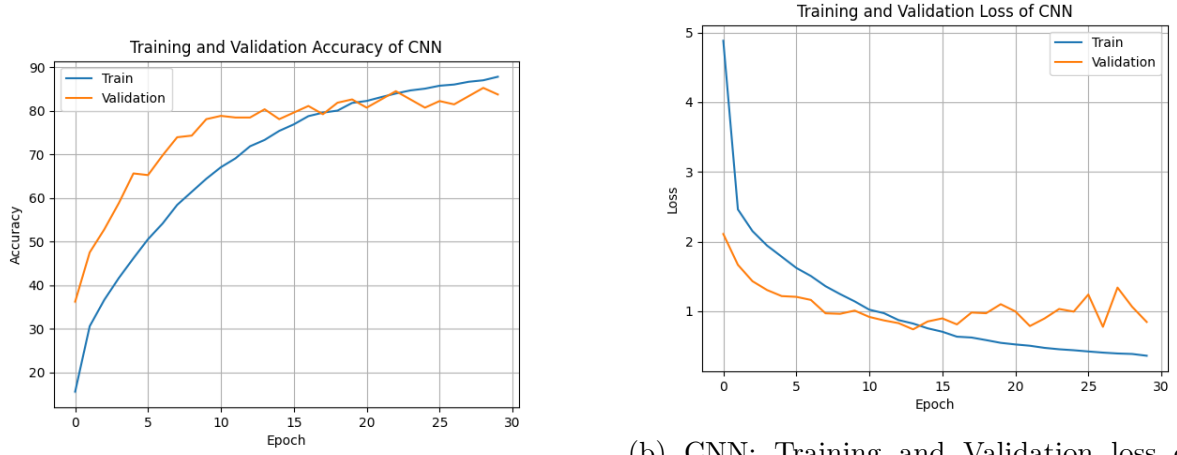
Task 1 (30 points):

1. Part 1 (10 points): CNN

The chosen dataset for this classification task is the Cards Image Dataset-Classification. This dataset is well-suited for image classification due to its high quality, consistent image dimensions ($224 \times 224 \times 3$ JPG), and pre-partitioned train, test, and validation sets across 53 distinct card classes. To manage computational cost and potentially aid feature extraction, the input images were intentionally blurred and resized to $112 \times 112 \times 3$ before being fed into the network. For the convolutional base, a pattern of three convolutional layers, each followed by Batch Normalization and ReLU activation, and then interleaved with two MaxPooling layers, was adopted. This design choice aims to progressively extract more abstract features while reducing spatial dimensions, a common and effective approach for image recognition. The kernel_size=3 and padding=1 maintain spatial resolution within convolutional blocks, focusing downsampling solely on MaxPooling. The final dense layers (1024 units then 53 units for classification) are connected after flattening the output of the last convolutional block, with a dropout layer introduced to mitigate overfitting.

The model was compiled and trained for 30 epochs, monitoring both training and validation loss and accuracy. The training results show a consistent decrease in training loss (from 4.8820 to 0.3614) and an increase in training accuracy (from 15.58% to 87.84%). On the validation set, the loss generally decreased (from 2.1088 to 0.8462) and accuracy improved (from 36.23% to 83.77%). The primary metrics used for evaluation are **Loss** (specifically cross-entropy loss, indicating the prediction error) and **Accuracy** (the proportion of correctly classified images). While the training metrics indicate good learning, the validation metrics, particularly accuracy, are crucial for assessing the model's generalization performance on unseen data. The validation accuracy of

83.77% suggests that the chosen CNN architecture and training regimen, including the use of blurred inputs, are reasonably effective for this card image classification task.



(a) CNN: Accuracy over epoch

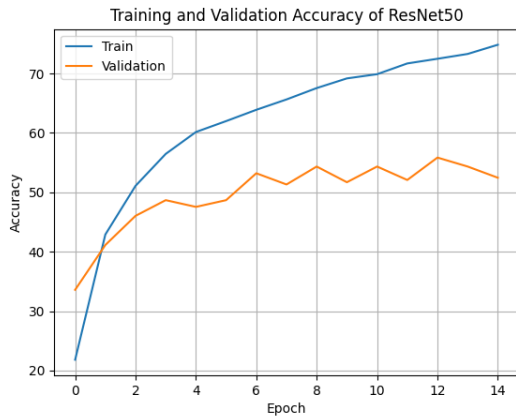
(b) CNN: Training and Validation loss over epoch

Figure 1: Convolutional Neural Network

2. Part 2 (10 points): ResNet50 (Transfer Learning and Finetuning)

For this classification task, ResNet50 was chosen as the pre-trained model for transfer learning, applying both feature extraction and fine-tuning. The final classification layer was adapted for 53 card classes, and models were trained for 15 epochs, fewer due to higher per-epoch computational costs than the custom CNN.

Comparing with the custom CNN’s 83.77% validation accuracy, both ResNet50 models performed lower (52.45% for feature extraction, 55.47% for fine-tuning). This discrepancy arises from several factors: fewer training epochs for ResNet50, potential domain mismatch between ImageNet and the card dataset, and notably, signs of overfitting in the ResNet50 models (high training accuracy vs. stagnant validation accuracy). Additionally, a critical difference might be the input image resolution: if the custom CNN utilized blurred images of $112 \times 112 \times 3$ to reduce computational load, while the ResNet50 models processed full $224 \times 224 \times 3$ images, this could significantly impact their performance and ability to generalize, as the pre-trained features might not align optimally with such high-resolution, specific inputs compared to a custom CNN trained from scratch on potentially reduced-resolution data.

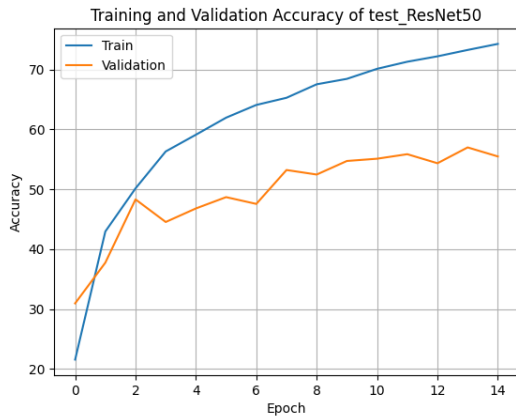


(a) ResNet50 Transfer Learning: Accuracy over epoch



(b) ResNet50 Transfer Learning: Training and Validation loss over epoch

(c) ResNet 50 Transfer Learning



(a) ResNet50 Finetuning: Accuracy over epoch



(b) ResNet50 Finetuning: Training and Validation loss over epoch

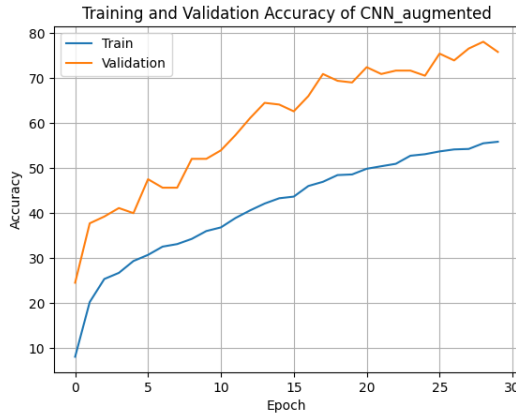
Figure 3: ResNet 50 Finetuning
*Mind the title of the figure

3. Part 3 (10 points): CNN with Data Augmentation

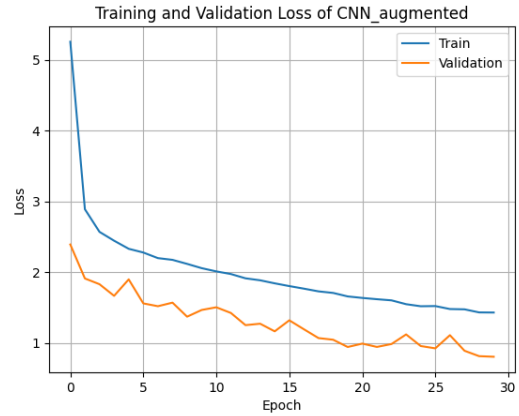
To enhance dataset diversity, data augmentation was applied to the Cards Image Dataset using `RandomHorizontalFlip()` and `RandomRotation(10)` transformations, in addition to the pre-existing blurring and resizing to 112×112 . The same custom CNN architecture from Part 1 was then trained on this augmented dataset for 30 epochs.

Upon repeating the training process with augmented data, a noticeable difference in results was observed. The final validation accuracy for the augmented dataset

was 75.85%, which is lower than the 83.77% achieved by the model trained on non-augmented (but blurred) data in Part 1. This unexpected decrease in performance could be attributed to the nature of the "Cards Image Dataset." Playing cards have specific orientations and symmetries. While horizontal flipping might be somewhat acceptable for some cards, random rotations, especially up to 10 degrees, could introduce significant distortions that make it harder for the model to correctly identify the card. For instance, rotating a "6 of Spades" could make it resemble a "9 of Spades" if not careful. The model might struggle to learn robust features when the augmented data presents orientations that are not typically seen or are confusing in the real-world context of card recognition. Essentially, while data augmentation generally helps prevent overfitting and improves generalization, in this specific domain, the chosen augmentations might have introduced noise or ambiguity that hindered the model's ability to learn precise distinguishing features, leading to a reduction in validation performance.



(a) CNN with Data Augmentation: Accuracy over epoch



(b) CNN with Data Augmentation: Training and Validation loss over epoch

Figure 4: Convolutional Neural Network with Data Augmentation

Task 2 (15 points):

1. Part 1 (7 points): CVAE

Data: Cat Faces Dataset

Upon implementing the Conditional Variational Autoencoder (CVAE) for image generation using a dataset of cat images, it became evident that the model struggled to capture the intricate details necessary for generating high-fidelity representations. The synthesized images were notably blurry, often presenting only a faint resemblance to actual cats. While the CVAE was able to learn the general shape and perhaps the presence of feline-like features, it largely failed to produce sharp edges, distinct fur patterns, or clear facial characteristics. Despite this blurriness, an encouraging observation was the variability in colors present in the generated images. This suggests that the CVAE was indeed learning and incorporating the diverse color distributions found in the training data, thereby indicating that the model was, at a fundamental level, grasping the existence and variability of "cats" within its learned latent space, even if the visual output lacked photorealism.



Figure 5: Generated Cat Images

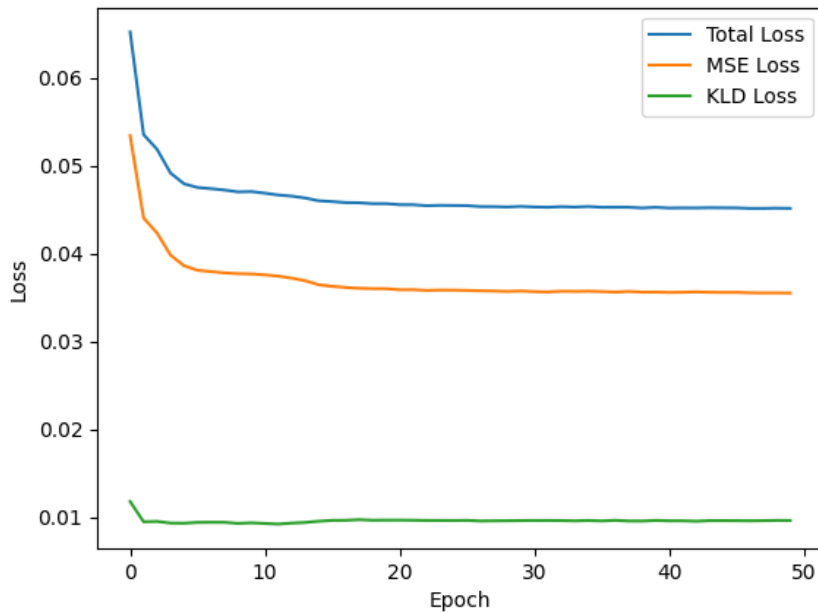


Figure 6: CVAE Loss

2. Part 2 (8 points): DCGAN

In contrast to the CVAE's output, the images generated by the Deep Convolutional Generative Adversarial Network (DCGAN) showed significantly more variability and vibrant colors, presenting a more promising and aesthetically pleasing result. The DCGAN had a superior ability to produce diverse feline-like images, often with sharper features and a wider range of poses and coat patterns than its VAE counterpart. However, this increased generative capacity came with a trade-off: while many generated images were convincing, a noticeable portion were nowhere near recognizable as cats, appearing as mere abstract smudges or random patterns. Compared to the CVAE, which tended to generate consistently blurry but vaguely cat-like forms, the DCGAN's strength lies in its ability to generate a wider spectrum of outputs, including highly realistic samples, but this greater freedom also allows for the generation of entirely random or uninterpretable artifacts.

Go to ipynb notebook for Generator learning progress over epochs.

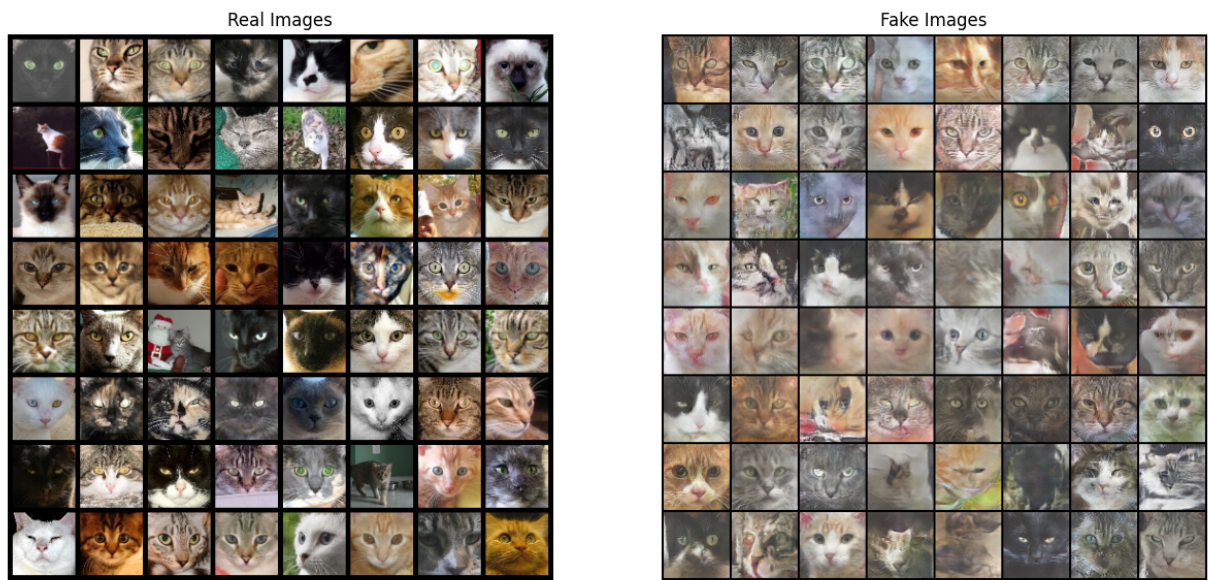


Figure 7: Real Images vs Generated Images

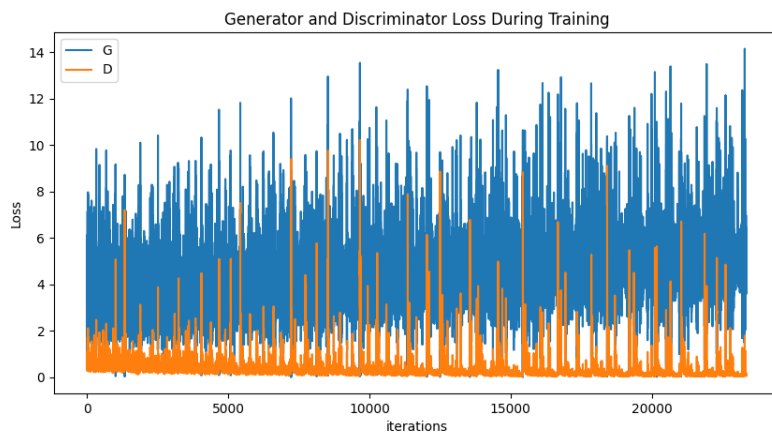


Figure 8: DCGAN Loss

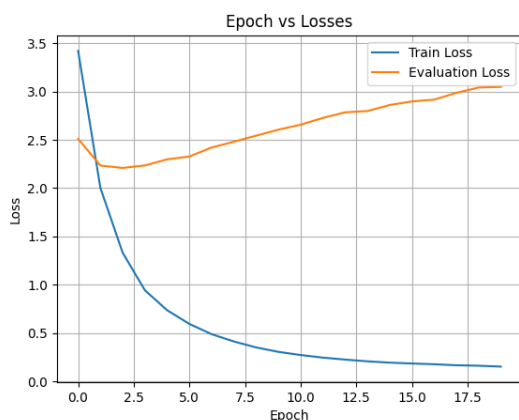
Task 3 (55 points): NLP and Attention Mechanism

1. **Part 1 (10 points):** Implement the scaled dot-product attention as discussed in class (lecture 14) from scratch (use NumPy and pandas only, no deep learning libraries are allowed for this step).
2. **Part 2 (10 points):** Pick any encoder-decoder seq2seq model (as discussed in class) and integrate the scaled dot-product attention in the encoder architecture.

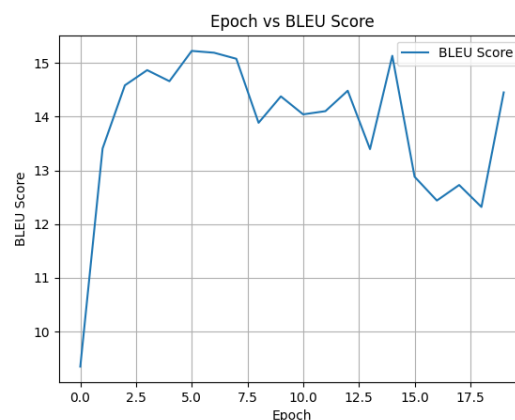
I chose to do LSTM based Encoder Decoder model.

3. **Part 3 (5 points):** Pick any public dataset of your choice (use a small-scale dataset like a subset of the Tatoeba or Multi30k dataset) for machine translation task. Train your model from Part 2 for the machine translation task. Evaluate test set by reporting the BLEU Score.

I chose Multi30k English to German.



(a) LSTM Encoder Decoder with Attention:
Loss over epoch



(b) LSTM Encoder Decoder with Attention:
BLEU Score over epoch

Figure 9: LSTM Encoder Decoder Model with Attention

Go to ipynb notebook for example generated outputs.

4. **Part 4 (30 points):** Implement Simple Transformer
 - (a) Reduced Model Depth: Use 2 encoder layers and 2 decoder layers instead of the standard 6.
 - (b) Limited Attention Heads: Use 2 attention heads in the multi-head attention mechanism rather than 8.
 - (c) Smaller Embedding Size: Set the embedding dimension to 64 instead of 512.
 - (d) Reduced Feedforward Network Size: Use a feedforward dimension of 128 instead of 2048.

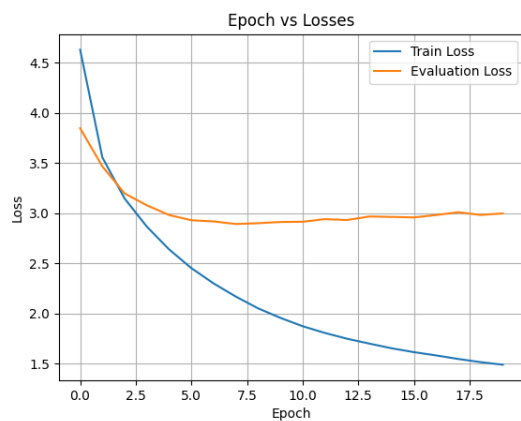
- (e) Smaller Dataset: Use a small dataset (e.g., about 10k sentence pairs).
- (f) Tokenization Simplifications: Use a basic subword tokenizer (like Byte Pair Encoding - BPE) or word-level tokenization instead of complex language-specific tokenizers.

Key components to implement:

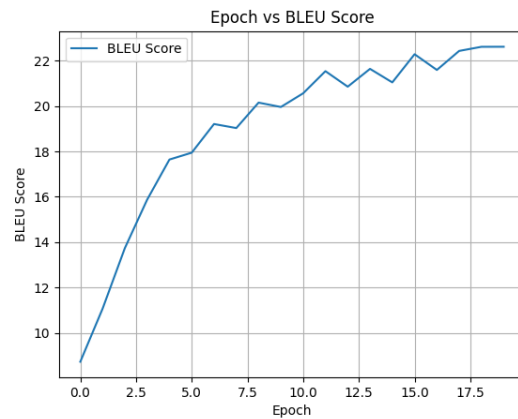
- (a) Positional Encoding: Implement Sinusoidal position encoding.
- (b) Scaled dot-product attention: Use the same implementation from part 1.
- (c) Multi-Head Attention: Integrate the scaled dot-product attention into a multi-head attention framework using the specified simplifications.
- (d) Encoder and Decoder Blocks: Implement simplified encoder and decoder layers, ensuring: Layer normalization, Residual connections, Masked attention in the decoder for autoregressive generation.
- (e) Final Output Layer: Implement a linear layer followed by a SoftMax activation for generating translated tokens.

The evaluation of machine translation models on the validation set reveals a significant performance gap between the attention-based Seq2Seq model (Part 2) and the simplified Transformer model (Part 4), with the latter achieving a notably higher BLEU score of 22.61 compared to the Seq2Seq’s best of 15.22. This difference in quality stems from the fundamental architectural advantages of the Transformer. Its multi-head attention mechanism allows for more efficient capture of long-range dependencies within sentences by processing tokens in parallel, a stark contrast to the sequential processing inherent in the LSTM layers of the Seq2Seq model. Furthermore, the Transformer’s explicit sinusoidal positional encoding provides a robust way to incorporate word order information, and its extensive use of layer normalization and residual connections aids in training deeper networks more effectively, contributing to better overall translation quality and generalization.

Beyond performance, a notable practical difference lies in runtime. The LSTM encoder-decoder with attention, despite its simpler appearance, often takes significantly longer per epoch to train. This is primarily due to its recurring architectural format, which necessitates processing input sequences one token at a time. This sequential dependency inherently limits parallelism during computation, making it less efficient on modern hardware compared to the Transformer. The Transformer, by design, processes entire sequences simultaneously, leveraging its attention mechanisms to compute relationships between all tokens in parallel. This parallelizability leads to considerably faster training times per epoch for the Transformer, even with its potentially larger number of parameters in a full-scale implementation, making it a more computationally efficient choice for machine translation tasks.



(a) Simplified Transformer: Loss over epoch



(b) Simplified Transformer: BLEU Score over epoch

Figure 10: Simplified Transformer

Go to [ipynb notebook](#) for example generated outputs.